ARM-450 — Command Reference

Every alias, every raw ROS 2 equivalent, and the commands worth knowing when something looks wrong.

This page is generated. The alias table below is parsed out of arm450_aliases.sh by make_commands.py; the trajectory options are read from plan_sine.py --help. Nothing here is typed by hand, because a hand-written command sheet is precisely the document that goes stale without anyone noticing.

1. Install, once

```
echo 'source ~/ros2_ws/arm450_design/arm450_aliases.sh' >> ~/.bashrc
exec bash
a450help          # the whole list, any time
```

If the workspace has never been built on this machine:

```
cd ~/ros2_ws
colcon build --packages-select arm450_description arm450_sine --symlink-install
source install/setup.bash
```

2. The short version

```
a450src           # source ROS 2 Jazzy + this workspace
a450run           # arm + gripper + sine trace in RViz
a450slow          # slow it down to watch the wrist
a450stop          # stop everything
```

Use a plain Ubuntu terminal. Inside the VS Code integrated terminal the snap sets GTK_PATH, RViz loads a GTK module from the snap tree, that module drags in the snap's glibc, and rviz2 dies on startup with

```
libpthread.so.0: undefined symbol: __libc_pthread_init, version GLIBC_PRIVATE
```

a450unsnap (unset GTK_PATH) fixes it. Outside the editor the variable is not set and the problem does not exist.

3. Every alias

Parsed from arm450_aliases.sh, so this table cannot drift from what your shell actually does.

environment

command	runs
a450src	source /opt/ros/jazzy/setup.bash && source $ARM450_WS/install/setup.bash && echo "ARM-450 env ready (ROS 2 jazzy)"

command	runs
a450build	cd $ARM450_WS && source /opt/ros/jazzy/setup.bash && colcon build --packages-select arm450_description arm450_sine && source $ARM450_WS/install/set…
a450cd	cd $ARM450_DIR

run / stop

command	runs
a450run	ros2 launch arm450_sine sine.launch.py
a450jog	ros2 launch arm450_description display.launch.py
a450stop	pkill -f sine_node; pkill -f robot_state_publisher; pkill -f rviz2; echo "ARM-450 nodes stopped"

speed (live, while it is running)

command	runs
a450speed()	a450speed 60 -> 60 Hz
a450slow	ros2 param set /arm450_sine rate 8.0
a450normal	ros2 param set /arm450_sine rate 25.0
a450fast	ros2 param set /arm450_sine rate 75.0
a450rate	ros2 param get /arm450_sine rate

shape (needs a re-solve, then relaunch)

command	runs
a450plan()	a450plan --amplitude 0.04 --cycles 4 --points 300
a450cycles4	a450plan --cycles 4 --points 300
a450cycles6	a450plan --cycles 6 --points 400
a450reset	a450plan --amplitude 0.04 --cycles 2 --span 0.14 --board-x 0.30 --z-center 0.22 --points 240
a450tipik	a450plan --board-x 0.30 --z-center 0.26 --w-ori 0.2 --tip-ik

inspect

command	runs
a450joints	ros2 topic echo /joint_states --once
a450hz	ros2 topic hz /joint_states
a450tcp	ros2 run tf2_ros tf2_echo base_link tcp
a450tip	ros2 run tf2_ros tf2_echo base_link tool_tip
a450nodes	ros2 node list
a450params	ros2 param list /arm450_sine

design / docs

command	runs
a450urdf()	a450urdf [tool] — regenerate BOTH urdfs and install them
a450tool()	a450tool gripper / dock / none
a450wrist	rviz2 -d $ARM450_WS/install/arm450_description/share/arm450_description/rviz/arm450_tool.rviz
a450unsnap()	unset GTK_PATH — lets rviz2 start in a VS Code snap terminal
a450gate	(cd $ARM450_DIR && python3 preprint_check.py)
a450flight	(cd $ARM450_DIR && python3 preflight.py)
a450cad	(cd $ARM450_DIR/cad && for f in *.py; do ["$f" = params.py] && continue; python3 "$f"; done)
a450docs	ls -lh $ARM450_DIR/output/*.pdf
a450cmds	(cd $ARM450_DIR && python3 make_commands.py && python3 build_pdf.py COMMANDS.md)
a450open	xdg-open $ARM450_DIR/output

Notes carried in the file

a450plan() — --tool matters. The trajectory drives the J6 FLANGE; the fitted tool's working point is further out (gripper 42 mm), so the board the TIP traces is 42 mm beyond --board-x. Passing --tool also moves the RViz path marker onto the tip, which is why the yellow curve used to float behind the fingers. ARM450_TOOL follows whatever a450tool last fitted.

a450cycles4 — a450cycles2 removed 2026-08-22: it was --cycles 2 --points 240, which is the shipped default, so it did exactly what a450reset does. Two names for one thing is

how they drift apart.

a450reset — z-center is 0.22, NOT 0.20. At 0.20 the exact-mesh check found the upper arm grazing the base pedestal on 30 % of waypoints. This alias still said 0.20 long after plan_sine.py was fixed, so a450reset would have quietly written a COLLIDING trajectory over the good one.

a450tipik — board fixed and cannot move? drive the TIP onto --board-x instead of the flange. Harder solve -- the flange ends up 42 mm nearer the base, on a poorer IK branch -- so it wants a higher trace and a stronger orientation weight.

a450tcp — tcp is the J6 FLANGE, not the working point. With a tool fitted that is the frame that hid the 42 mm gap, so it is labelled and a450tip is the one to use.

a450urdf() — Rebuilds BOTH urdfs. It used to rebuild only arm450.urdf (the box-primitive one); the mesh urdf RViz actually loads was left stale, which is how RViz came to be showing a model that did not match the design.

a450tool() — swap the fitted end effector and rebuild in one go

a450wrist — camera parked on the wrist, so the tool is actually visible

a450unsnap() — rviz2 dies instantly in a VS Code snap terminal with libpthread.so.0: undefined symbol: __libc_pthread_init, GLIBC_PRIVATE The cause is GTK_PATH, which the snap points at its own GTK module directory. rviz2's Qt/GTK integration loads a module from there and that module drags in the snap's core20 glibc alongside the system one. Nothing else matters -- LD_LIBRARY_PATH, PATH and LOCPATH are all red herrings; unsetting this one variable is the whole fix.

a450cmds — regenerate the command reference. It is PARSED from this file and from plan_sine.py --help, so editing an alias and re-running this keeps the PDF honest -- which is the whole reason it is generated rather than typed.

4. Environment variables

variable	value
ARM450_WS	~/ros2_ws
ARM450_DIR	~/ros2_ws/arm450_design
ARM450_TRAJ	~/ros2_ws/src/arm450_sine/sine_traj.npz
ARM450_TOOL	gripper

ARM450_TOOL follows whatever a450tool last fitted, and a450plan passes it to plan_sine.py automatically.

5. Raw ROS 2 — the same things without aliases

Useful when you are on another machine, or debugging and want to see exactly what is being run.

Launch

```
source /opt/ros/jazzy/setup.bash
```

```
source ~/ros2_ws/install/setup.bash

ros2 launch arm450_sine sine.launch.py                 # trace, with RViz
ros2 launch arm450_sine sine.launch.py traj_file:=/path/to/other.npz
ros2 launch arm450_description display.launch.py       # RViz + joint sliders
ros2 launch arm450_description display.launch.py gui:=false
```

Run a single node

```
ros2 run arm450_sine sine_node --ros-args \
    -p traj_file:=$HOME/ros2_ws/src/arm450_sine/sine_traj.npz \
    -p approach_time:=3.0 \
    -p loop_mode:=pingpong

URDF=~/ros2_ws/install/arm450_description/share/arm450_description/urdf
ros2 run robot_state_publisher robot_state_publisher --ros-args \
    -p robot_description:="$(cat $URDF/arm450_meshes.urdf)"

RVIZ=~/ros2_ws/install/arm450_description/share/arm450_description/rviz
rviz2 -d $RVIZ/arm450.rviz
```

Three RViz configs ship: arm450.rviz (whole arm), arm450_tool.rviz (camera on the wrist), arm450_side.rviz (looks along −Y, so board distance and tip are unambiguous — perspective in the orbit views makes the traced curve look like it passes through the gripper when it does not).

Parameters, live

```
ros2 param list /arm450_sine
ros2 param get  /arm450_sine rate
ros2 param set  /arm450_sine rate 60.0      # 0.5 – 500 Hz, takes effect at once
ros2 param dump /arm450_sine
```

rate is the only parameter that can be changed while running and have an effect. create_timer() fixes its period at construction, so the node rebuilds the timer in a parameter callback — without that, ros2 param set would be accepted and then silently ignored.

Topics

```
ros2 topic list
ros2 topic echo /joint_states --once
ros2 topic hz   /joint_states               # should sit at the `rate` param
ros2 topic info /arm450/trajectory --verbose
ros2 topic echo /arm450/trajectory --once    # the whole solved path, latched
ros2 topic echo /sine_path --once            # the marker RViz draws
```

/joint_states is a visualisation stream — a real controller will not follow it. /arm450/trajectory is the same path as a JointTrajectory with timestamps and velocities, published once and latched (TRANSIENT_LOCAL), which is what a FollowJointTrajectory action wants.

Frames

```
ros2 run tf2_ros tf2_echo base_link tool_tip     # where the tool actually works
ros2 run tf2_ros tf2_echo base_link tcp          # the J6 flange, 42 mm behind it
ros2 run tf2_tools view_frames                   # writes frames.pdf
ros2 topic echo /tf_static --once
```

tcp is not the working point. It is the J6 tool face; a fitted gripper works 42 mm further out at tool_tip. Reading tcp and assuming it was the pen is what put the traced curve 42 mm behind the fingers.

Recording and replay

```
ros2 bag record /joint_states /sine_path /tf /tf_static -o arm450_trace
ros2 bag info arm450_trace
ros2 bag play arm450_trace --loop
```

Inspecting the model

```
check_urdf $URDF/arm450_meshes.urdf
urdf_to_graphiz $URDF/arm450_meshes.urdf     # writes arm450.gv / .pdf
ros2 node list
ros2 node info /arm450_sine
ros2 doctor --report
```

6. Re-solving the trajectory

The ROS node replays a solved file; it does not do IK. Changing amplitude, cycles, span or board distance needs plan_sine.py, then a relaunch.

```
usage: plan_sine.py [-h] [--amplitude AMPLITUDE] [--cycles CYCLES]
                    [--span SPAN] [--board-x BOARD_X] [--points POINTS]
                    [--z-center Z_CENTER] [--tool {dock,gripper,none,pen}]
                    [--tip-ik] [--w-ori W_ORI] [--out OUT]

Re-solve the ARM-450 sine trajectory. Changing amplitude or cycles REQUIRES
this step -- the ROS node only replays a solved file, it does not do IK.

options:
  -h, --help            show this help message and exit
  --amplitude AMPLITUDE
                        sine amplitude in m (default 0.04)
  --cycles CYCLES       number of full sine cycles (default 2.0)
  --span SPAN           trace length along the board in m (default 0.14)
  --board-x BOARD_X     board distance from base in m (default 0.3)
  --points POINTS       waypoints (default 240); more = smoother, slower solve
  --z-center Z_CENTER   trace centre height in m (default 0.22). THE lever for
                        large amplitude: at 0.20 the shoulder J2 sits on its
                        +115 deg stop, so amplitude past 40 mm degrades. 0.17
                        frees it -- 80 mm amplitude solves to 0.39 mm rms
                        there vs 1.99 mm at 0.20.
  --tool {dock,gripper,none,pen}
                        fitted end effector. Its working point is past the J6
                        flange (gripper 42 mm, dock 36, pen 30), so the board
                        the TIP traces is that much further out than --board-x
```

```
                      unless you also pass --tip-ik.
  --tip-ik            drive the TOOL TIP onto --board-x instead of the
                      flange. Use when the board cannot move. Harder solve
                      -- try --z-center 0.26 --w-ori 0.2 with a gripper.
  --w-ori W_ORI       orientation weight (default 0.05). Raise it with
                      --tip-ik: a tilt moves the tip, so at 0.05 the solver
                      trades 21 deg of tilt for position error.
  --out OUT
```

The board distance changes with a tool fitted

The trajectory drives the J6 flange. A fitted tool works further out, so the board the tip traces is that much beyond --board-x:

fitted	works past the flange	board goes at
none	—	300 mm
pen	30 mm	330 mm
gripper	42 mm	342 mm
dock probe	36 mm	336 mm

plan_sine.py prints the number so you never work it out:

```
  TOOL: gripper, working point 42 mm past the J6 flange
  IK drives the FLANGE
  *** PUT THE BOARD AT x = 342 mm ***  (flange sweeps x = 300 mm)
```

If the board cannot move, a450tipik drives the tip onto 300 mm instead. It is the worse option — 2.87° joint steps against 2.09°, and 0.087 mm rms against 0.055 — because the flange ends up 42 mm nearer the base on a poorer IK branch. It also needs --z-center 0.26 and --w-ori 0.2: with a tool fitted a tilt moves the tip, so at the flange-tuned weight the solver trades 21° of tool tilt for position error.

Keep amplitude at 0.04. Past 40 mm the shoulder J2 reaches its +115° stop and path error goes 0.19 → 1.35 mm. Cycles are free — raise --points to about 60 × cycles and the error does not move.

7. Design-side commands

```
cd ~/ros2_ws/arm450_design

python3 preflight.py           # the full 8-stage pre-print check  (a450flight)
python3 preprint_check.py      # the fast parts gate               (a450gate)
python3 check_tool_fit.py      # end-effector interfaces + the docking mate
python3 check_sine_clear.py    # sine path vs exact mesh, per fitted tool
python3 workspace.py           # reach and swept volume, per fitted tool
python3 interference.py        # joint-limit collision sweep

python3 cad/end_effector.py    # rebuild the six tool parts
python3 cad/volumes.py         # refresh the exact STEP volumes
python3 draw_iso.py            # isometric sheets   -> figures/iso/
python3 draw_3view.py          # third-angle sheets -> figures/3view/
```

```
python3 build_pdf.py REPORT.md  # any .md -> output/ARM450_*.pdf
```

Swapping the fitted end effector

```
a450tool gripper     # or: dock | none
```

That regenerates both URDFs, copies the meshes, colcon-builds, and re-solves the trajectory so the path marker lands on the new tool's tip. Then a450run.

Manually:

```
cd ~/ros2_ws/arm450_design
EMIT=1 TOOL=dock python3 generate_urdf_meshes.py
cp meshes/*.stl ~/ros2_ws/src/arm450_description/meshes/
cp arm450_meshes.urdf ~/ros2_ws/src/arm450_description/urdf/
cd ~/ros2_ws && colcon build --packages-select arm450_description --symlink-install
```

8. When something looks wrong

symptom	cause	fix
rviz2 exits instantly, GLIBC symbol error	VS Code snap sets GTK_PATH	a450unsnap, or use a plain terminal
RViz shows blocks, not the real arm	loaded arm450.urdf (primitives) instead of arm450_meshes.urdf	use the launch file, or a450urdf
meshes missing, arm is invisible	package:// name wrong — it is arm450_description, never arm450_design	rebuild with a450urdf
no motion in RViz	sine_node not running, or Fixed Frame is not world	a450nodes, check the RViz Global Options
arm jumps back to the start each pass	loop_mode:=wrap	leave it at pingpong
traced curve sits behind the gripper	path marker on the flange	re-solve with --tool, see §6
a450stop kills your terminal	you used pkill -f ros2	a450stop names each executable for this reason

Stopping things safely

```
a450stop        # pkill -f sine_node; pkill -f robot_state_publisher; pkill -f rviz2
```

Never pkill -f ros2. The pattern matches your own shell's command line and kills the terminal you are typing in. For the same reason, a wait loop built on pgrep -f <script> never exits — it matches itself.

9. Reference PDFs

file	what is in it
ARM450_RUN.pdf	running the simulation, in full

file	what is in it
ARM450_URDF.pdf	the kinematic chain, every link and joint
ARM450_BUY.pdf	what to buy, what to print, print order
ARM450_PREFLIGHT.pdf	the 8-stage pre-print gate
ARM450_END_EFFECTORS.pdf	the gripper and the docking probe
ARM450_WORKSPACE.pdf	reach and swept volume
ARM450_DRAWINGS_ISO.pdf	dimensioned isometric sheets, 18 parts
ARM450_DRAWINGS_3VIEW.pdf	third-angle sheets, 18 parts
ARM450_LOG.pdf	what changed, when, and why
ARM450_LESSONS.pdf	the mistakes, and the rules they produced

a450docs lists them; a450open opens the folder.